SIMATS ENGINEERING
ASSESSMENT TOOL 2

COURSE NAME: INTERNET PROGRAMMING
COURSE CODE: CSA43

COURSE OUTCOME COVERED

CO -2 : Apply CSS layout and styling techniques to create visually appealing and responsive web interfaces, and use JavaScript to enable interactive client-side behavior. (L3)

Assessment Tool : Design Task

Weightage: 20%

Code of Conduct:

I Narravula Navya Sri - 192324220 *(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*



Signature of the student:

QUESTIONS: **Total Marks: 20**

Question 1: Responsive Navigation Bar Design - Marks(4)

Suppose that an e-commerce website requires a responsive navigation bar that adapts across desktop, tablet, and mobile devices.
(a) Identify key components required in a navigation bar for an e-commerce website.
(b) Design a responsive navigation bar using appropriate CSS techniques.
(c) Demonstrate how the navigation adapts to smaller screens (e.g., hamburger menu).
(d) Evaluate the usability and suggest improvements for better user experience.

Question 2: Product Hover Effect - Marks(4)

Suppose that an e-commerce website wants to enhance user interaction through visual effects on product items.
(a) Identify suitable CSS properties for hover effects.
(b) Suggest appropriate **CSS animation or transition** for product interaction.
(c) Design a hover effect for a product card.
(d) Evaluate performance and user experience considerations.

Question 3: Layout Selection (Flexbox vs Grid) - Marks(4)

Suppose that a dashboard layout needs to be designed for a web application.
(a) Identify layout requirements of a dashboard (rows, columns, alignment).
(b) Compare **Flexbox and Grid** based on layout needs.
(c) Choose the most suitable layout technique and justify your choice.
(d) Evaluate the scalability and responsiveness of the chosen approach.

Question 4: Mobile-First Blog Layout - Marks(4)

Suppose that a blog page must be designed following a mobile-first approach.
(a) Identify key sections of a blog layout.
(b) Design a mobile-first layout using CSS.
(c) Demonstrate how the layout scales for larger screens using media queries.
(d) Evaluate accessibility and readability of the design.

Question 5: JavaScript Event Handling for Dynamic Menu - Marks(4)

Suppose that a website includes a dynamic menu that responds to user interactions.
(a) Identify appropriate JavaScript events for menu interaction.
(b) Design event handling logic for menu toggle functionality.
(c) Implement interaction for user actions (click/hover).
(d) Evaluate performance and suggest improvements.

1: Responsive Navigation Bar Design

(a) Key Components of an E-commerce Navigation Bar

The important components of an e-commerce navigation bar are:

1. **Logo:** Represents the website's brand.
2. **Navigation links:** Links to Home, Products, Categories, About, and Contact pages.
3. **Search bar:** Helps users find products quickly.
4. **Shopping cart:** Displays selected products.
5. **Login/Profile:** Allows users to access their accounts.
6. **Hamburger menu:** Displays navigation links on smaller screens.

(b) Design a Responsive Navigation Bar Using CSS

```
<!DOCTYPE html>
<html>
<head>
<style>
nav {
  display: flex;
  justify-content: space-between;
  align-items: center;
  background: #222;
  padding: 15px;
  color: white;
}
nav a {
  color: white;
  text-decoration: none;
  margin: 10px;
}
.menu {
  display: flex;
}
.hamburger {
  display: none;
  font-size: 25px;
  cursor: pointer;
}
@media (max-width: 768px) {
  .menu {
    display: none;
    flex-direction: column;
    position: absolute;
    top: 55px;
    right: 0;
```

```
    background: #222;
    width: 100%;
  }

  .menu.active {
    display: flex;
  }
  .hamburger {
    display: block;
  }
}
</style>
</head>
<body>
<nav>
  <h3>ShopNow</h3>
  <span class="hamburger" onclick="toggleMenu()">☰</span>
  <div class="menu" id="menu">
    <a href="#">Home</a>
    <a href="#">Products</a>
    <a href="#">Categories</a>
    <a href="#">Cart</a>
  </div>
</nav>
<script>
function toggleMenu() {
  document.getElementById("menu").classList.toggle("active");
}
</script>
</body>
</html>
```

(c) How the Navigation Adapts to Smaller Screens

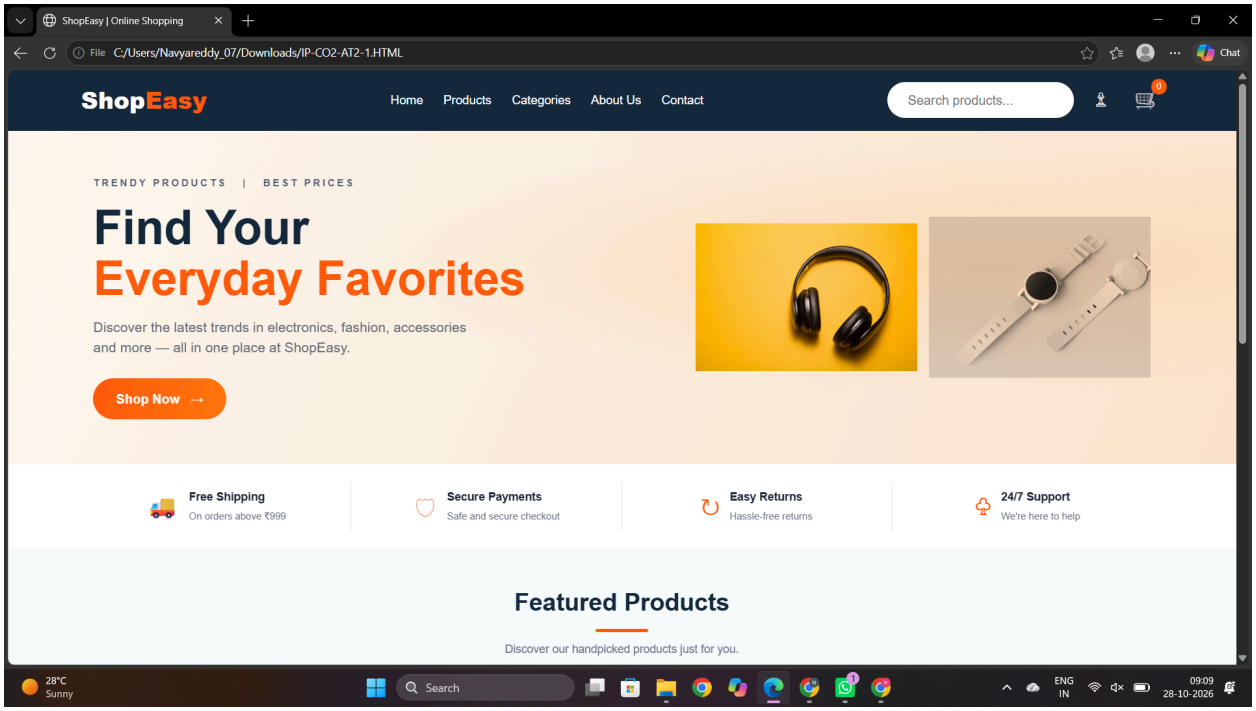
- **Desktop:** Navigation links appear horizontally.
- **Tablet:** The navigation adjusts to the available screen width.
- **Mobile:** The navigation links are hidden, and a hamburger icon appears. Clicking the icon displays the menu vertically.

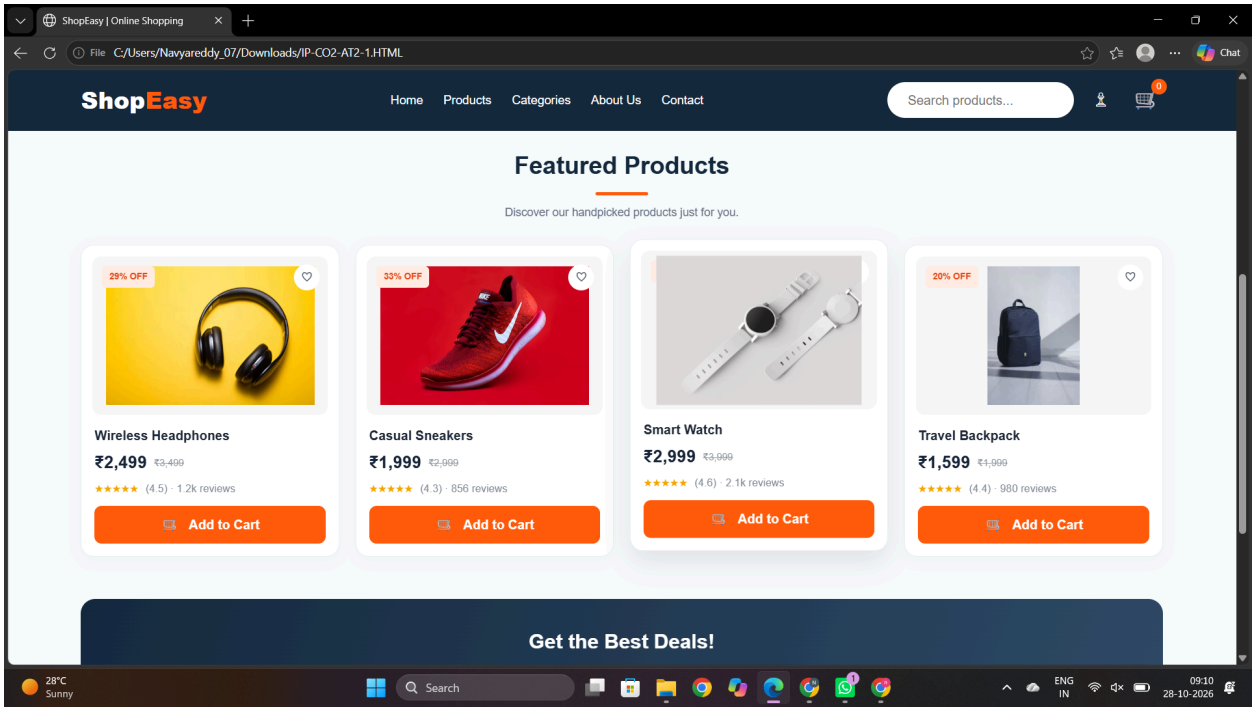
(d) Usability Evaluation and Improvements

A responsive navigation bar provides easy access to important pages on different devices. The hamburger menu saves space on mobile screens.

Improvements:

- Use clear and readable labels.
- Add keyboard navigation and visible focus indicators.
- Provide an accessible name and expanded state for the hamburger button.
- Ensure sufficient spacing between clickable elements.
- Close the menu after a link is selected on mobile.





Question 2: Product Hover Effect

(a) Suitable CSS Properties for Hover Effects

The following CSS properties are suitable for product hover effects:

1. transform – Moves, scales, or rotates an element.
2. box-shadow – Adds a shadow to the product card.
3. opacity – Changes the transparency of an element.
4. background-color – Changes the background colour.
5. transition – Creates smooth changes between styles.

(b) Appropriate CSS Animation or Transition

A smooth transition can be used to enlarge the product card slightly and add a shadow when the user moves the mouse over it.

Example:

```
.product {
  transition: transform 0.3s ease,
              box-shadow 0.3s ease;
}
.product:hover {
  transform: scale(1.05);
  box-shadow: 0 6px 15px rgba(0, 0, 0, 0.2);
}
```

(c) Design a Hover Effect for a Product Card

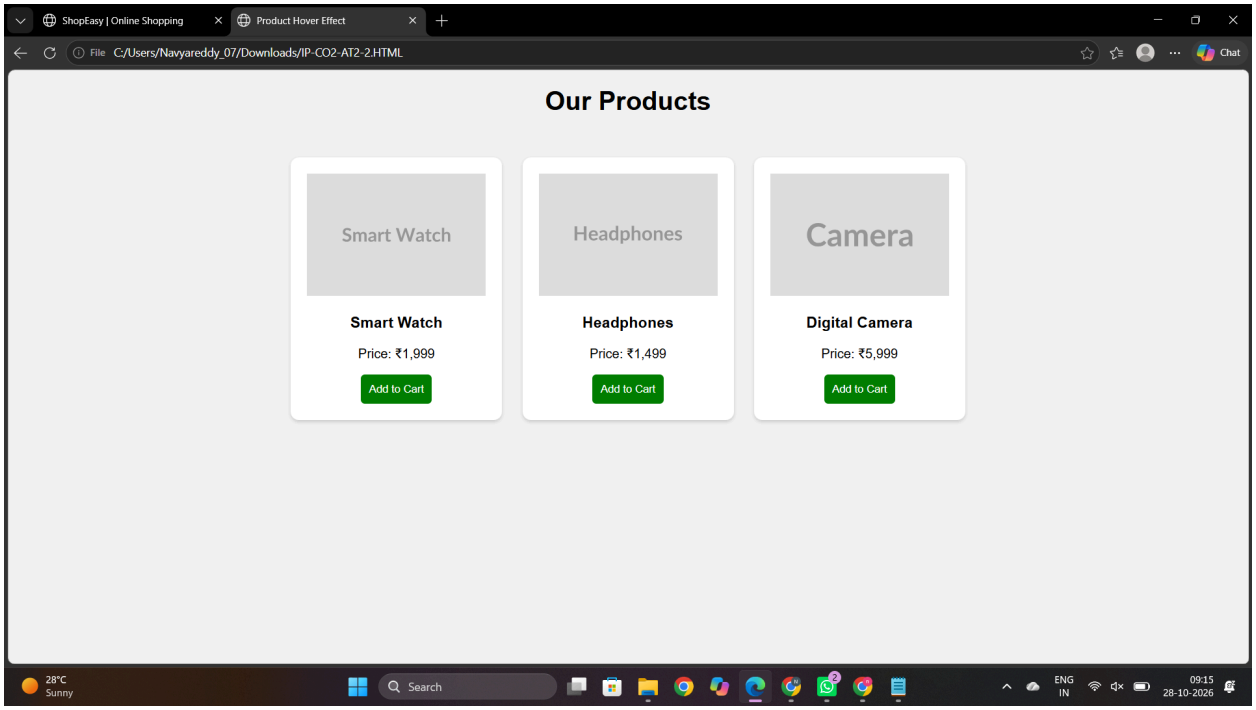
```
<!DOCTYPE html>
<html>
<head>
<style>
.product {
  width: 220px;
  padding: 15px;
  text-align: center;
  border: 1px solid #ddd;
  border-radius: 8px;
  transition: transform 0.3s ease,
              box-shadow 0.3s ease;
}
.product img {
  width: 100%;
  border-radius: 5px;
```

```
}
.product:hover {
  transform: translateY(-5px);
  box-shadow: 0 8px 16px rgba(0, 0, 0, 0.2);
}
button {
  background: green;
  color: white;
  padding: 10px;
  border: none;
  cursor: pointer;
}
</style>
</head>
<body>
<div class="product">
  <img src="product.jpg" alt="Product image">
  <h3>Smart Watch</h3>
  <p>Price: ₹1,999</p>
  <button>Add to Cart</button>
</div>
</body>
</html>
```

Output: When the user hovers over the product card, it moves slightly upward and displays a shadow, making the product more noticeable.

(d) Performance and User Experience Considerations

- Use transform and opacity for smoother animations.
- Avoid excessive animations that distract users.
- Keep transition durations short, around 200–300 milliseconds.
- Use hover effects as enhancements rather than the only way to reveal information.
- Support touchscreens and keyboard users.
- Respect the prefers-reduced-motion setting to reduce animations for users who prefer less motion.



Question 3: Layout Selection (Flexbox vs Grid)

(a) Layout Requirements of a Dashboard

A dashboard generally contains:

1. **Header:** Displays the title, profile, and notifications.
2. **Sidebar:** Contains navigation links.
3. **Main content:** Displays charts, tables, and reports.
4. **Cards:** Show important statistics.
5. **Rows and columns:** Organize information clearly.
6. **Responsive design:** Adapts to desktop, tablet, and mobile screens.

(b) Comparison Between Flexbox and Grid

Feature	Flexbox	CSS Grid
Layout type	One-dimensional	Two-dimensional

Direction	Row or column	Rows and columns
Alignment	Excellent for aligning items in one direction	Excellent for aligning items in both directions
Best use	Navigation bars and button groups	Dashboards and complex page layouts
Responsiveness	Supports flexible item arrangement	Supports responsive rows and columns
Control	Controls items along one main axis	Controls the overall layout structure

(c) Most Suitable Layout Technique and Justification

CSS Grid is the most suitable technique for a dashboard because dashboards usually require multiple rows and columns to display charts, cards, tables, and other information.

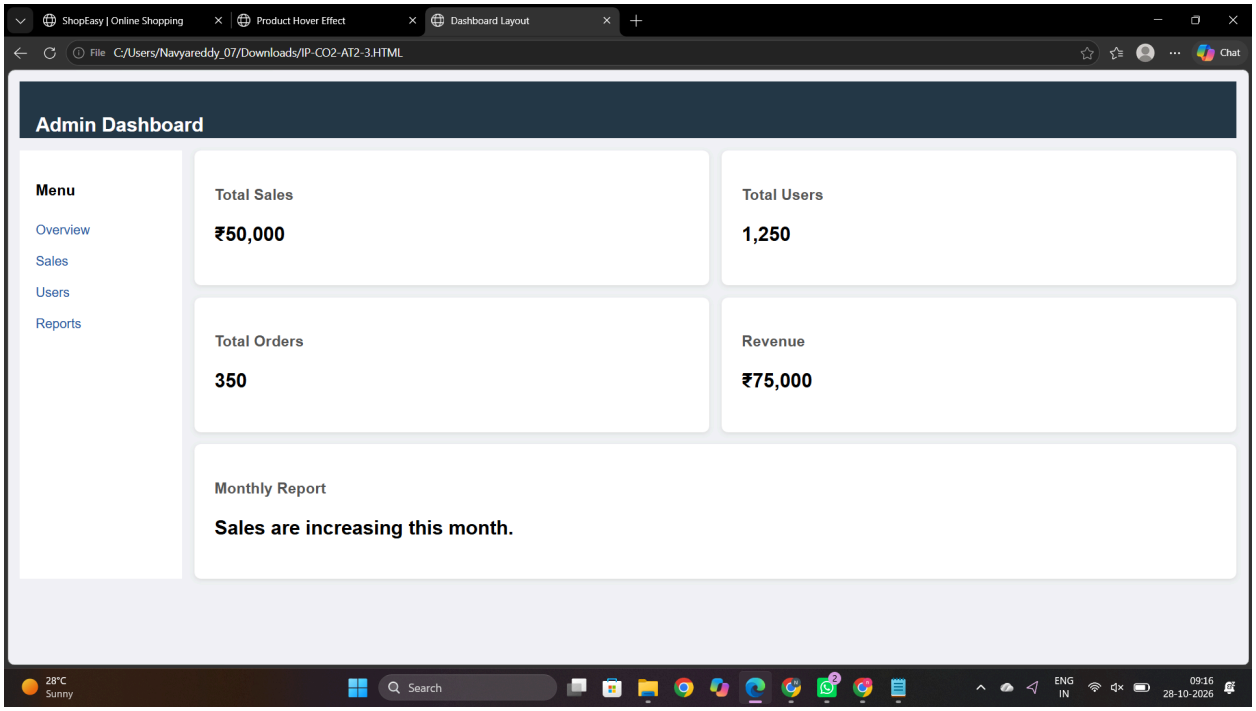
Example:

```
<div class="dashboard">
  <header>Dashboard Header</header>
  <aside>Sidebar</aside>
  <main>
    <div class="card">Total Sales</div>
    <div class="card">Total Users</div>
    <div class="card">Revenue</div>
    <div class="card">Reports</div>
  </main>
</div>
```

(d) Scalability and Responsiveness

CSS Grid makes it easy to add or remove dashboard cards without redesigning the entire layout. Media queries allow the dashboard to change from multiple columns on desktop to a single column on mobile.

Conclusion: CSS Grid is best for the overall dashboard structure, while Flexbox can be used inside individual components for aligning buttons, icons, and text.



Question 4: Mobile-First Blog Layout

(a) Key Sections of a Blog Layout

A typical blog page contains:

1. **Header:** Displays the blog name or logo.
2. **Navigation bar:** Provides links to different sections.
3. **Main content:** Displays blog posts and articles.
4. **Sidebar:** Contains categories, recent posts, or related links.
5. **Footer:** Displays copyright information and contact links.

(b) Design a Mobile-First Layout Using CSS

A mobile-first approach means designing for smaller screens first and then adding styles for larger screens.

```
<!DOCTYPE html>
<html>
<head>
<style>
* {
  box-sizing: border-box;
}
body {
  margin: 0;
  font-family: Arial, sans-serif;
  line-height: 1.6;
}
header, footer {
  background: #333;
  color: white;
  text-align: center;
  padding: 15px;
}
.container {
  padding: 15px;
}
article, aside {
  background: #f4f4f4;
  padding: 15px;
  margin-bottom: 15px;
  border-radius: 5px;
}
article img {
```

```
  max-width: 100%;
  height: auto;
}
a {
  color: #0645ad;
}
@media (min-width: 768px) {
  .container {
    display: grid;
    grid-template-columns: 2fr 1fr;
    gap: 20px;
    max-width: 1100px;
    margin: auto;
  }
}
</style>
</head>
<body>
<header>
  <h1>My Blog</h1>
</header>
<div class="container">
  <main>
    <article>
      <h2>Benefits of Web Design</h2>
      <p>Responsive web design improves usability across different devices.</p>
    </article>
    <article>
```

```
    <h2>Learning CSS</h2>
    <p>CSS helps create attractive and responsive web pages.</p>
  </article>
 </main>
 <aside>
  <h3>Recent Posts</h3>
  <p>HTML Basics</p>
  <p>CSS Tutorials</p>
 </aside>
</div>
<footer>
 <p>&copy; 2026 My Blog</p>
</footer>
</body>
</html>
```

(c) How the Layout Scales for Larger Screens

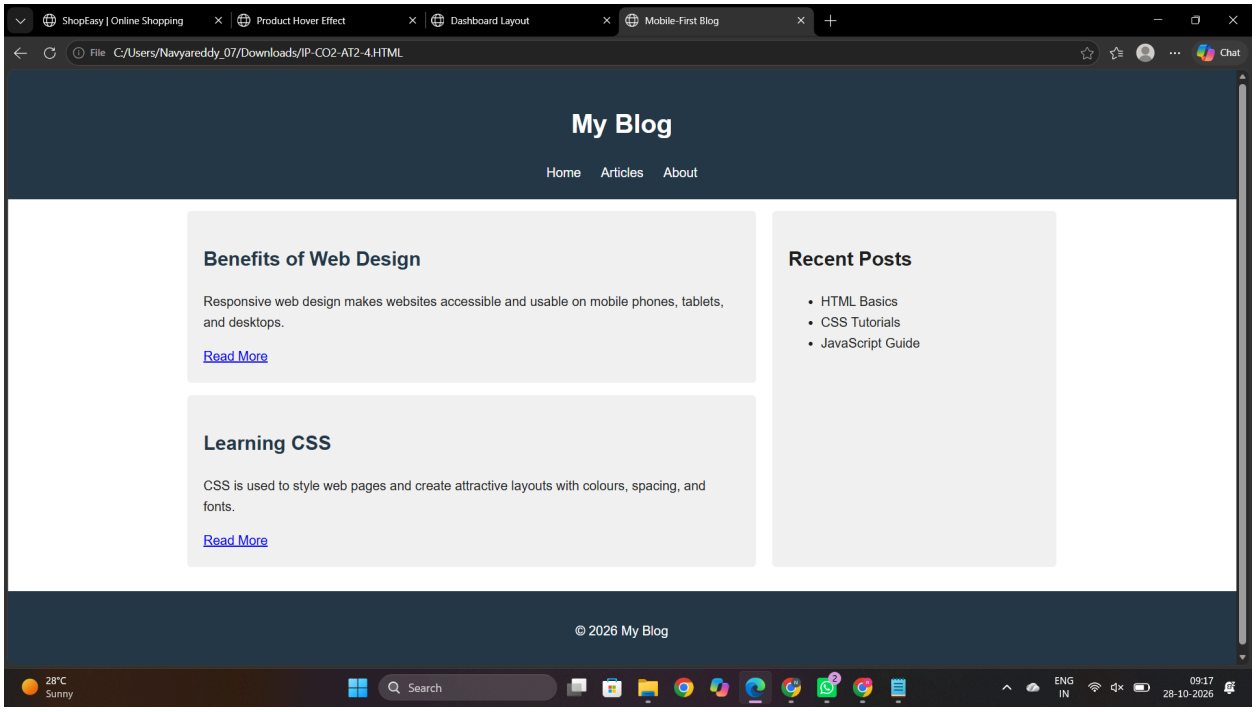
- **Mobile (below 768px):** Blog articles and sidebar appear vertically in a single column.
- **Tablet and desktop (768px and above):** CSS Grid displays the main content and sidebar in two columns.
- Images automatically adjust to the available width using max-width: 100%.

The media query ensures that the layout adapts to different screen sizes.

(d) Accessibility and Readability

- Use readable fonts and sufficient font sizes.
- Maintain good colour contrast between text and background.
- Use semantic HTML elements such as <header>, <main>, <article>, and <footer>.
- Provide descriptive alternative text for meaningful images.
- Use clear headings and appropriate paragraph spacing.
- Ensure links are keyboard-accessible and have visible focus indicators.

Conclusion: A mobile-first blog layout improves readability, accessibility, and usability across different devices.



Question 5: JavaScript Event Handling for Dynamic Menu

(a) Appropriate JavaScript Events for Menu Interaction

The following events can be used:

1. **click:** Opens or closes the menu when the user clicks a button.
2. **mouseover:** Detects when the mouse moves over an element.
3. **mouseout:** Detects when the mouse leaves an element.
4. **keydown:** Detects keyboard input, such as the Escape key.
5. **DOMContentLoaded:** Runs JavaScript after the HTML document has been loaded and parsed.

(b) Event Handling Logic for Menu Toggle Functionality

A menu toggle function changes the menu's visibility when the user clicks the hamburger button. The classList.toggle() method adds or removes a CSS class.

(c) Implement Click and Hover Interactions

```
<!DOCTYPE html>
<html>
<head>
<style>
.menu {
  display: none;
  background: #333;
  padding: 10px;
}
.menu.active {
  display: block;
}
.menu a {
  display: block;
  color: white;
  padding: 10px;
  text-decoration: none;
}
.menu a:hover,
.menu a:focus-visible {
  background: #555;
}
button {
  padding: 10px;
  cursor: pointer;}
</style>
```

```
</head>
<body>
<button id="menuButton"
        aria-expanded="false"
        aria-controls="menu">
</button>
<nav id="menu" class="menu" aria-label="Main navigation">
  <a href="#home">Home</a>
  <a href="#products">Products</a>
  <a href="#about">About</a>
  <a href="#contact">Contact</a>
</nav>
<script>
const button = document.getElementById("menuButton");
const menu = document.getElementById("menu");
button.addEventListener("click", function () {
  const isOpen = menu.classList.toggle("active");
  button.setAttribute("aria-expanded", isOpen);});
document.addEventListener("keydown", function (event) {
  if (event.key === "Escape") {
    menu.classList.remove("active");
    button.setAttribute("aria-expanded", "false");
    button.focus();}});
</script>
</body>
</html>
```

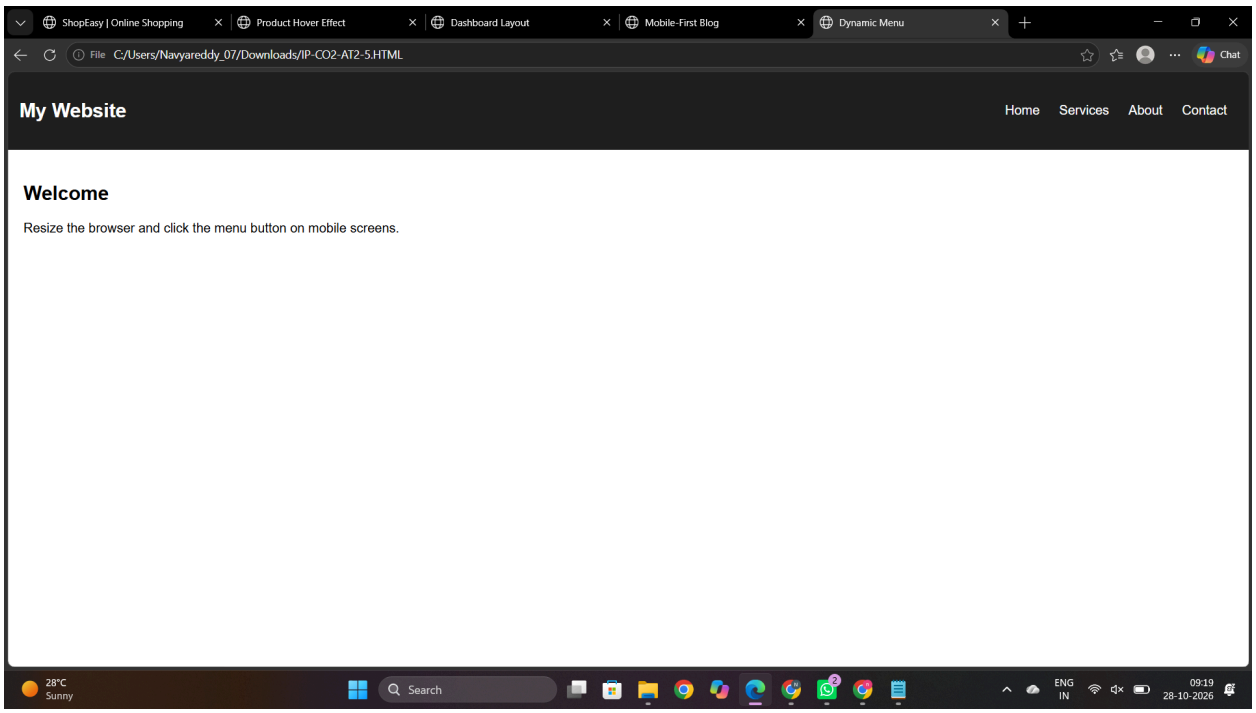
Explanation:

- The click event opens or closes the menu.
- The classList.toggle() method switches the active class.
- The aria-expanded attribute informs assistive technologies whether the menu is open.
- The keydown event closes the menu when the user presses Escape.
- The :hover and :focus-visible selectors highlight links during mouse or keyboard interaction.

(d) Performance Evaluation and Improvements

- Use addEventListener() to handle events cleanly.
- Avoid unnecessary event listeners and repeated DOM updates.
- Prefer CSS for simple hover effects instead of JavaScript.
- Use event delegation when handling events for many menu items.
- Ensure keyboard accessibility and support for touchscreen devices.
- Keep the JavaScript simple and avoid unnecessary animations.

Conclusion: JavaScript event handling makes the navigation menu interactive, while CSS controls its appearance and responsive behaviour.



RUBRICS FOR CALCULATION:

Criteria	Weightage (%)	Level 4 – Exemplary (4)	Level 3 – Proficient (3)	Level 2 – Developing (2)	Level 1 – Beginning (1)
Understanding of Concept	25	Demonstrates complete understanding of design requirements and concepts	Good understanding with minor gaps	Basic understanding with some errors	Poor understanding or incorrect concepts
Design / Implementation	30	Well-structured, efficient, and responsive design implementation	Correct implementation with minor issues	Basic implementation with inconsistencies	Incorrect or incomplete implementation
Use of Tools (CSS/JS)	20	Effective and advanced use of tools/techniques	Appropriate use of tools	Limited or inconsistent use	Incorrect or minimal use
Analysis & Evaluation	15	Insightful analysis with strong justification and optimization	Good analysis with justification	Basic analysis with limited depth	Poor or no analysis
Documentation / Clarity	10	Clear, well-structured, and properly presented solution	Mostly clear with minor issues	Basic clarity, missing details	Poor or unclear documentation